SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Faculty of Computing

Year 2 Semester 1 (2026)

IT2011 - Artificial Intelligence and Machine Learning

Lab Sheet 07

Building an Artificial Neural Network (ANN)

Objective

The goal of this lab is to design, train, and evaluate an Artificial Neural Network (ANN) to predict student performance (Pass/Fail) using multiple academic, demographic, and behavioral features.

By the end of this lab, you will:

- Perform preprocessing (handle categorical + numerical data).
- Build and train an ANN model using TensorFlow/Keras.
- Evaluate model performance using accuracy, confusion matrix, and classification report.
- Experiment with different hyperparameters to improve performance.

Dataset Overview

File: student_success_expanded_balanced.csv
Rows: 951
Columns: 21

Sample Features:

- Age (numeric)
- Gender (categorical: Male/Female)
- Country (categorical)

- AttendanceRate (0–1 normalized attendance fraction)
- StudyHoursPerWeek (numeric)
- SleepHours (numeric)
- InternetSpeedMbps (numeric)
- DeviceType (categorical: Laptop/Mobile/Desktop)
- PreviousGPA (numeric, 0–4 scale)
- QuizScore, AssignmentScore, MidtermScore, FinalScore (numeric, 0–100 scale)

Target: Result → Pass or Fail

Step 1: Data Preparation

1. Load and explore the dataset with Pandas.

- Check .shape, .head(), .isnull().sum(), .describe().
- Look at value counts of the target: df['Result'].value_counts().

2. Preprocessing:

- Encode categorical variables (Gender, Country, DeviceType, Tutoring, ParentalEducation, LearningStyle) using **One-Hot Encoding**.
- Convert target column Result → 1 for Pass, 0 for Fail (Label Encoding).
- Scale numeric features (StandardScaler or MinMaxScaler).

3. Split dataset:

- Train = 80%, Test = 20%
- Use train_test_split(stratify=y) to keep balance of Pass/Fail.

Checkpoint: After this step, you should have training and test sets ready for ANN input.

Step 2: Build ANN Model

Import Keras

```
from tensorflow import keras
from tensorflow.keras import layers
```

Define architecture (simple starter model):

```
model = keras.Sequential([
    layers.Input(shape=(num_features,)),       # number of processed features
    layers.Dense(32, activation="relu"),       # Hidden Layer 1
    layers.Dense(16, activation="relu"),       # Hidden Layer 2
    layers.Dense(1, activation="sigmoid")      # Output layer
])
```

Compile model

```
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

Checkpoint: Print model.summary() and note the number of trainable parameters.

Step 3: Train the Model

Train with:

```
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=1
)
```

Plot training vs validation curves:

- Loss over epochs
- Accuracy over epochs

Checkpoint: Look at plots. Did the model overfit (val accuracy drops while train accuracy rises)?

Step 4: Evaluate the Model

1. Evaluate on test data:

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)
```

2. Generate predictions and confusion matrix:

```
from sklearn.metrics import confusion_matrix,
classification_report
import seaborn as sns
import matplotlib.pyplot as plt

y_pred = (model.predict(X_test) > 0.5).astype("int32")

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

print(classification_report(y_test, y_pred))
```

Checkpoint: Compare accuracy with class balance. Is the model biased toward one class?

Step 5: Experimentation (Optional)

- Change number of neurons/layers (e.g., 64-32-16).
- Try activation functions: relu, tanh, sigmoid.
- Add **Dropout** layers (e.g., layers.Dropout(0.3)) to reduce overfitting (if model overfits).
- Adjust **learning rate** in Adam optimizer.